When you clone the Rideshare application and "migrate" (a verb in Rails speak) the database, you've evolved your database schema to be in sync with the latest production version.

Now that you have an idea of what the Rails application is, how the database structure evolves and how to get a copy of the structure, you'll want some data.

You'll use a couple of ways to get data into your database. To make it easy to get started, you can use a pre-made data dump that can be downloaded and loaded.

Then you'll learn how to populate even more data via scripts that are part of the Rideshare codebase.

It's time to get Rideshare set up locally so you can explore the source code and start the database server and web server.

Installing Rideshare

Begin by installing the dependencies needed to run Rideshare. Once the dependencies are installed you'll install the Rideshare application.

The examples here were developed on Mac OS, using the latest released versions of Ruby on Rails and PostgreSQL at the time of writing; these are listed below.

- Mac OS Ventura
- Ruby on Rails 7 (Released 2021)
- PostgreSQL 15 (Released 2022)

Run the `rails` executable that's part of the application by running `bin/rails` from the root directory of Rideshare. You'll use `bin/rails` to run commands like `console`, `server`, and `runner`. You might see instructions to install Rails using `gem install rails`. This method will make a `rails` executable available, but don't use this version for these exercises.

If you're an experienced Rails application developer and PostgreSQL user, you may be able to skip the next few sections.

If you're less experienced with Ruby on Rails or with PostgreSQL, need a refresher, or prefer to make sure everything is installed upfront correctly, go through the instructions below to get everything configured on your local development computer.

By the end you'll have Rideshare fully running locally for development, connected to a PostgreSQL database server instance.